

Microinstruction Control: The Mic-1

Memory in the Mic-1

Two ways to access memory

The Mic-1 accesses the same underlying memory in **two different ways**, depending on whether it is accessing data or fetching an instruction byte:

1. A **word-oriented** data access, used for ordinary data memory, reading and writing 32-bit words through `MAR` (address) and `MDR` (data)
2. A **byte-oriented** instruction fetch, used to fetch `I JVM` instruction bytes one at a time through `PC` and `MBR`

Why two access modes?

The JVM instruction set is *byte oriented*:

- opcodes and operand bytes are stored one byte after another so that programs remain compact.

But the Mic-1 data path and ALU operate primarily on **32-bit words**.

Reading and writing **complete words** is therefore convenient for stack values, local variables, and other data.

So the Mic-1 provides two views of memory:

- a **word-oriented view** for ordinary data access
- a **byte-oriented view** for instruction fetching

These are not two separate physical memories. They are two different ways of accessing the same memory system.

Converting between the two views

The Mic-1's **MAR** contains a **word address**, while instruction fetching uses the byte-oriented **PC**.

Because each word contains 4 bytes, converting a word address to the corresponding byte address requires multiplying by 4.

In hardware, this is equivalent to shifting the address left by 2 bits:

```
1  byte address = word address × 4
2                = word address << 2
```

If **MAR** = 3, the corresponding byte address is 12 (3 × 4).

A normal **READ** operation does **not** require four separate byte reads. It reads an entire 32-bit word and places that word into **MDR**.

A **FETCH**, on the other hand, reads a single byte and places it into **MBR**.

Microinstructions

What is a microinstruction?

A **microinstruction** is one row of the control store:

a 36-bit control word that, for a single clock cycle, specifies

- which register drives the B bus,
- which registers latch the C bus,
- what function the ALU and shifter perform,
- whether memory should be read, written, or fetched, and
- where control should go next.

one ISA instruction, such as `ADD`, is implemented by executing a **sequence of microinstructions**,

with each microinstruction controlling the datapath for one clock cycle.

Memory operations take time

Memory operations initiated by a microinstruction are not immediately available to the datapath.

For example, when a microinstruction requests a memory `READ`, the requested word is not immediately available in `MDR`.

In the standard Mic-1 timing model, the result of a memory read initiated during microinstruction `k` becomes available for use in approximately `microinstruction k + 2`.

Therefore, the microprogram must leave enough time between requesting a memory value and using that value.

If microinstruction `k` performs `READ`, a later microinstruction must wait for the memory system to place the requested word into `MDR` before using that value.

The $k + 2$ rule and memory operations

A useful rule for Mic-1 microprogramming is:

A memory read requested in microinstruction k is safely usable in microinstruction $k + 2$.

This delay exists because memory access takes time relative to the datapath's internal operations.

The Mic-1's memory-control field provides three operations:

- **READ** - read a word from memory into **MDR**
- **WRITE** - write the word in **MDR** to memory
- **FETCH** - fetch a byte from memory into **MBR**

A microinstruction normally asserts **at most one** of these memory operations.

This keeps the memory-control logic simple and avoids attempting incompatible memory operations during the same cycle.

The decoder

The Mic-1 contains a **4-to-16 decoder** associated with the **B** field of the microinstruction.

The **B** field contains 4 bits, which are decoded into one of 16 possible selection lines.

Only **9 of those 16 encodings** correspond to actual B-bus sources:

- MDR
- PC
- MBR
- MBRU
- SP
- LV
- CPP
- TOS
- OPC

The remaining seven encodings are unused.

The decoder therefore allows a compact 4-bit field in the microinstruction to select which register drives the B bus.

Microinstruction Control: The Mic-1

How the machine decides which control signals should be enabled on each cycle

This is determined by something called the **sequencer**.

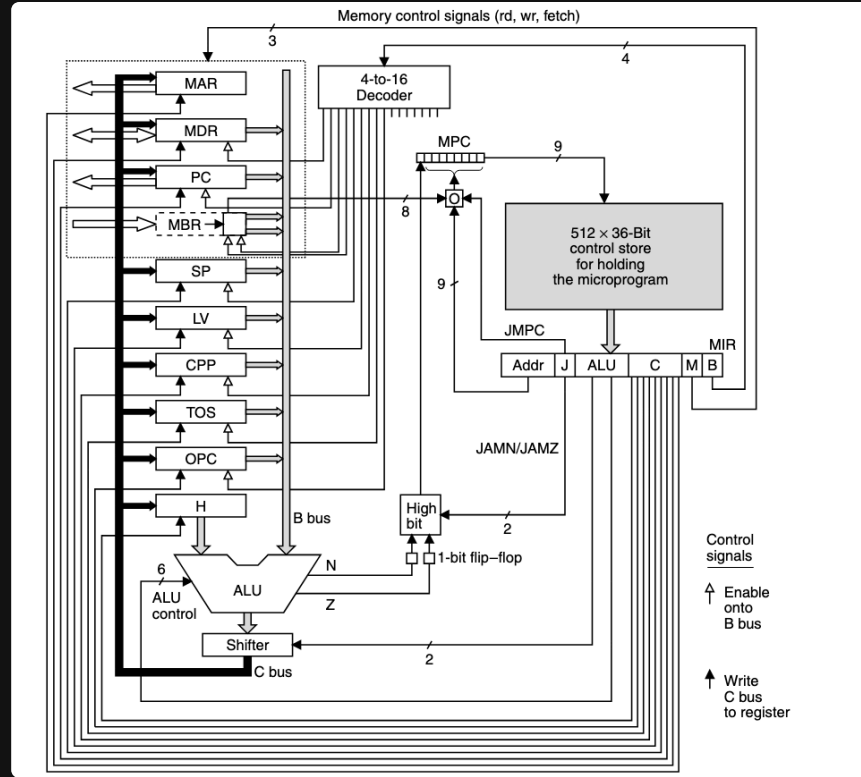
The sequencer steps through a sequence of microinstructions to implement the execution of a single ISA instruction.

For each cycle, it determines two major things:

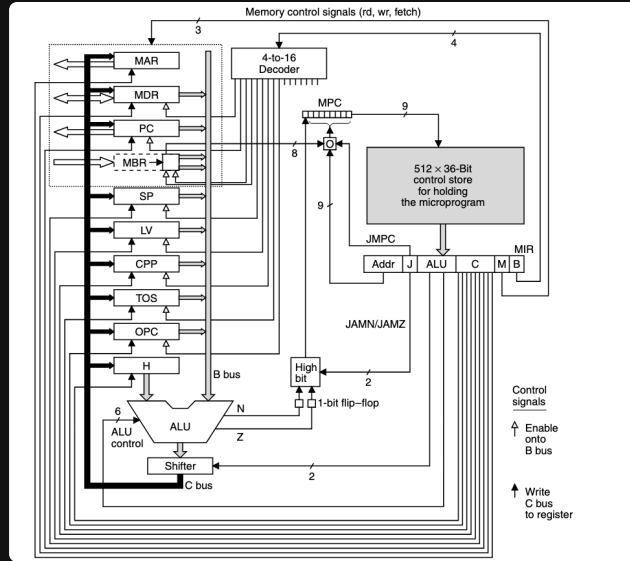
1. The state of the machine's control signals
2. The address of the microinstruction that should execute next

The control signals configure the datapath, ALU, registers, and memory system, while the next address determines how the microprogram proceeds.

The Mic-1 Architecture

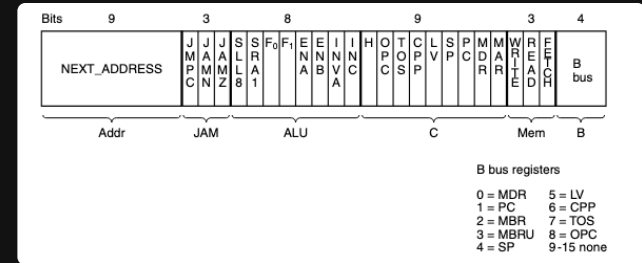


The Mic-1 Architecture



On the right, there is something called the **control store**, which is a memory that holds the complete microprogram.

For the Mic-1, the control store contains **512 words**, with each word being a **36-bit microinstruction**.



Reading the field layout

The 36-bit microinstruction word is divided into several named fields, from left to right:

Field	Width	Purpose
NEXT_ADDRESS	9 bits	Base address of the next microinstruction
JAM (JMPC , JAMN , JAMZ)	3 bits	Controls whether the next address is modified using MBR , N , and/or Z
ALU (SLL8 , SRA1 , F0 , F1 , ENA , ENB , INVA , INC)	8 bits	Selects the ALU function and shift/increment operations

Reading the field layout

Field	Width	Purpose
C	9 bits	Selects which registers latch the C-bus result
Mem (WRITE , READ , FETCH)	3 bits	Selects the memory operation
B	4 bits	Selects which register drives the B bus

The control store has 512 possible addresses because the microaddress is 9 bits wide:

$$2^9 = 512$$

This provides enough locations to hold the microprogram implementing the IJVM instruction set.

The MPC

One important difference between **ISA instructions** and *microinstructions* is that microinstructions do **not necessarily execute sequentially**.

Microinstruction routines frequently need to branch or jump to different parts of the microprogram.

The control store therefore uses two important registers:

1. The **Microprogram Counter (MPC)**: holds the address of the next microinstruction to be read from the control store
2. The **Microinstruction Register (MIR)**: holds the current microinstruction read from the control store

Why the MPC needs to jump around

Unlike the ISA-level `PC` , which normally advances through the program's instruction bytes, the `MPC` frequently changes to different locations in the microprogram.

For example:

- After an IJVM opcode has been fetched into `MBR` , control must jump to the microcode routine associated with that opcode
- Within a microprogram routine, control may branch depending on the result of an ALU operation
- Conditional operations can use the `N` and `Z` flags to determine which microinstruction executes next

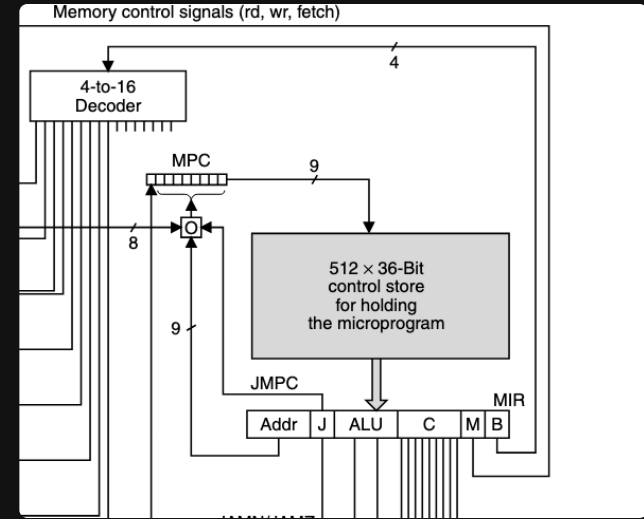
The microcode for an instruction such as `IF_ICMPEQ` can use the `Z` flag to determine whether the comparison produced zero and therefore whether the branch condition was satisfied.

The **JAM bits** control how the next MPC value is formed.

The MIR

It holds:

1. **NEXT_ADDRESS** and **JAM**: control how the address of the next microinstruction is determined
2. **ALU**: contains the 8 bits that select the ALU function, increment operation, and shift operations
3. **C**: selects which registers receive the ALU result from the C bus
4. **Mem**: controls the memory operation — **READ**, **WRITE**, or **FETCH**
5. **B**: selects which register supplies the B bus



Why is the B field 4 bits?

The B field uses 4 bits to select among 9 possible B-bus sources:

- MDR
- PC
- MBR
- MBRU
- SP
- LV
- CPP
- TOS
- OPC

Four bits provide 16 possible encodings, so seven encodings are unused.

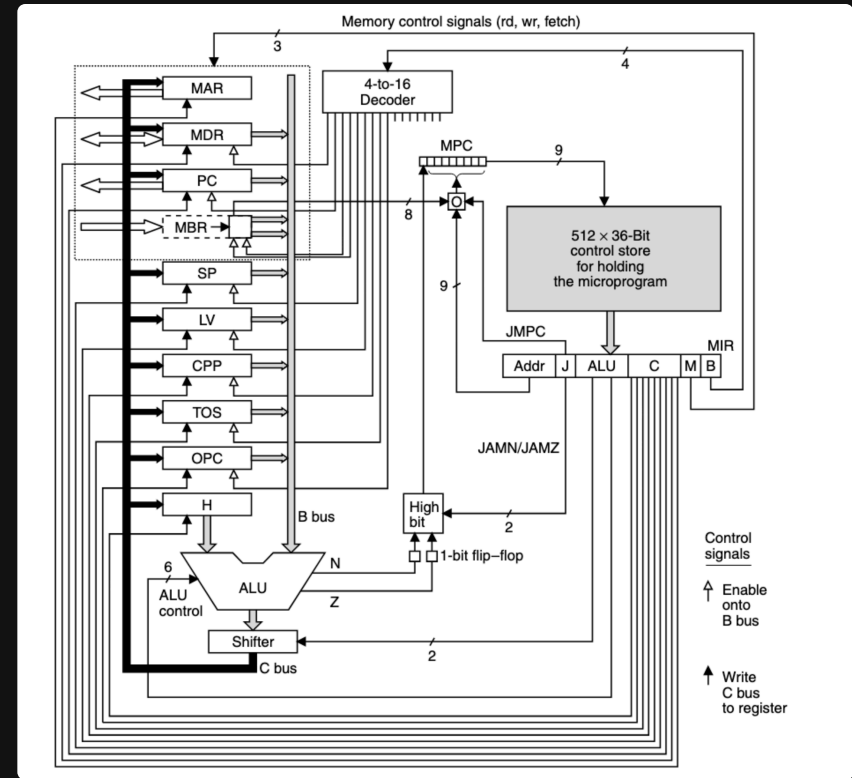
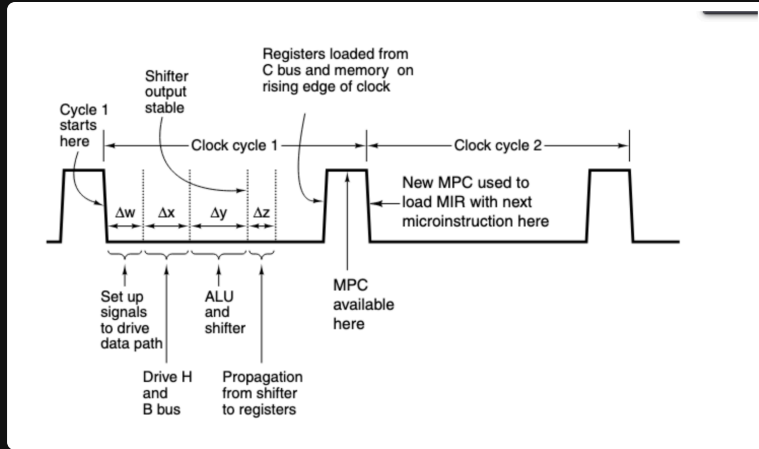
A more compact encoding could theoretically eliminate some of this unused space, but the Mic-1 uses the straightforward 4-bit selection scheme and a 4-to-16 decoder.

Note: MBR and MBRU refer to two different ways of placing the same 8-bit MBR value onto the 32-bit B bus:

- MBR sign-extends the byte
- MBRU zero-extends the byte

This distinction matters because some IJVM operands represent signed values while others should be treated as unsigned byte values.

Operation Sequence



Reading the timing diagram

Within a **single clock cycle**, the timing diagram breaks the datapath activity into several propagation delays that must all fit before the next clock edge:

- Δw - control signals from the MIR begin propagating through the datapath
- Δx - the **H** register and B-bus sources settle
- Δy - the ALU and shifter compute their result
- Δz - the result propagates toward the registers that will receive the C-bus value

The important idea is that the datapath has a **critical path**:

the signals must propagate through all of these stages quickly enough for the destination registers to capture the result at the next clock edge.

Summary

1. MPC provides the address of the next microinstruction
2. Control store is accessed using the MPC
3. MIR receives the microinstruction
4. MIR fields propagate into the datapath and control logic
5. B field selects the B-bus source
6. ALU field selects the ALU and shifter operations
7. ALU computes its result
8. C field selects which registers receive the C-bus result
9. Mem field controls READ , WRITE , or FETCH
10. N and Z are determined from the ALU result
11. JAM and NEXT_ADDRESS determine the next MPC value
12. Selected registers latch the C-bus result at the clock edge
13. The new MPC begins the next microinstruction cycle

In parallel

Several of these operations occur during the same microinstruction cycle.

While the datapath is computing the current microinstruction's result:

- The next MPC value is being determined
- The JAM logic examines N, Z, and/or MBR as appropriate
- The memory system performs the requested memory operation
- The C-bus result propagates toward its destination registers

The microinstruction therefore specifies a collection of control signals that operate **in parallel**, rather than being a simple list of sequential operations.

Putting the whole cycle together

Tracing through one full microinstruction cycle end-to-end:

1. MPC holds the address of the microinstruction to execute.
2. That address is used to access the **control store**.
3. The selected 36-bit microinstruction is loaded into MIR .
4. Once the MIR contents propagate, its fields simultaneously configure the datapath and memory system:
 - B selects the B-bus source
 - ALU selects the ALU/shifter operation
 - C selects the registers that will latch the C-bus result
 - Mem selects READ , WRITE , FETCH , or no memory operation

Putting the whole cycle together

5. The ALU receives its inputs from the `H` register and B bus and computes the requested result.
6. The result propagates onto the C bus.
7. The ALU result determines the `N` and `Z` flags.
8. At the clock edge, the selected registers latch the C-bus value.
9. At the same time, the next MPC value is loaded based on `NEXT_ADDRESS` and the `JAM` logic.
10. The new MPC then identifies the microinstruction for the next cycle.